

Introduction to Computing Systems

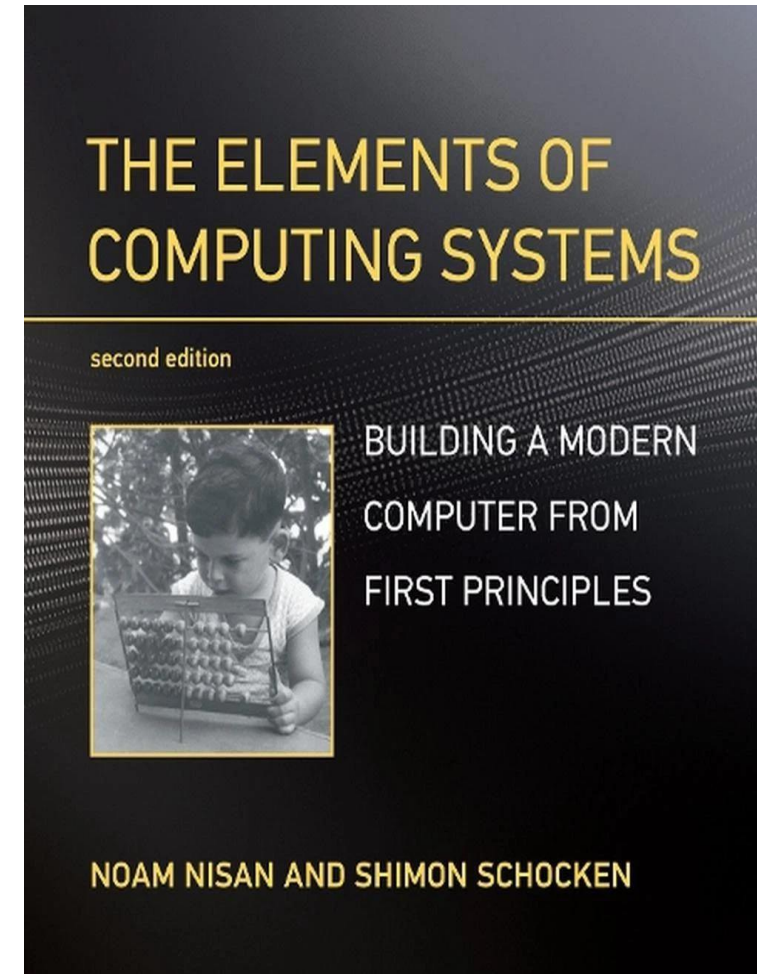
What This Course is About

- Hardware + Software combined in one journey
- Build a computer from scratch, step by step
- Understand the “big picture” of computing systems
- Learn both theory + practical implementation

Course Textbook

The Elements of Computing Systems: Building a Modern Computer From First Principles

- Authors: Noam Nisan & Shimon Schocken
- Also known as **Nand2Tetris**
- Free resources: website, tools, projects, emulator
- www.nand2tetris.org

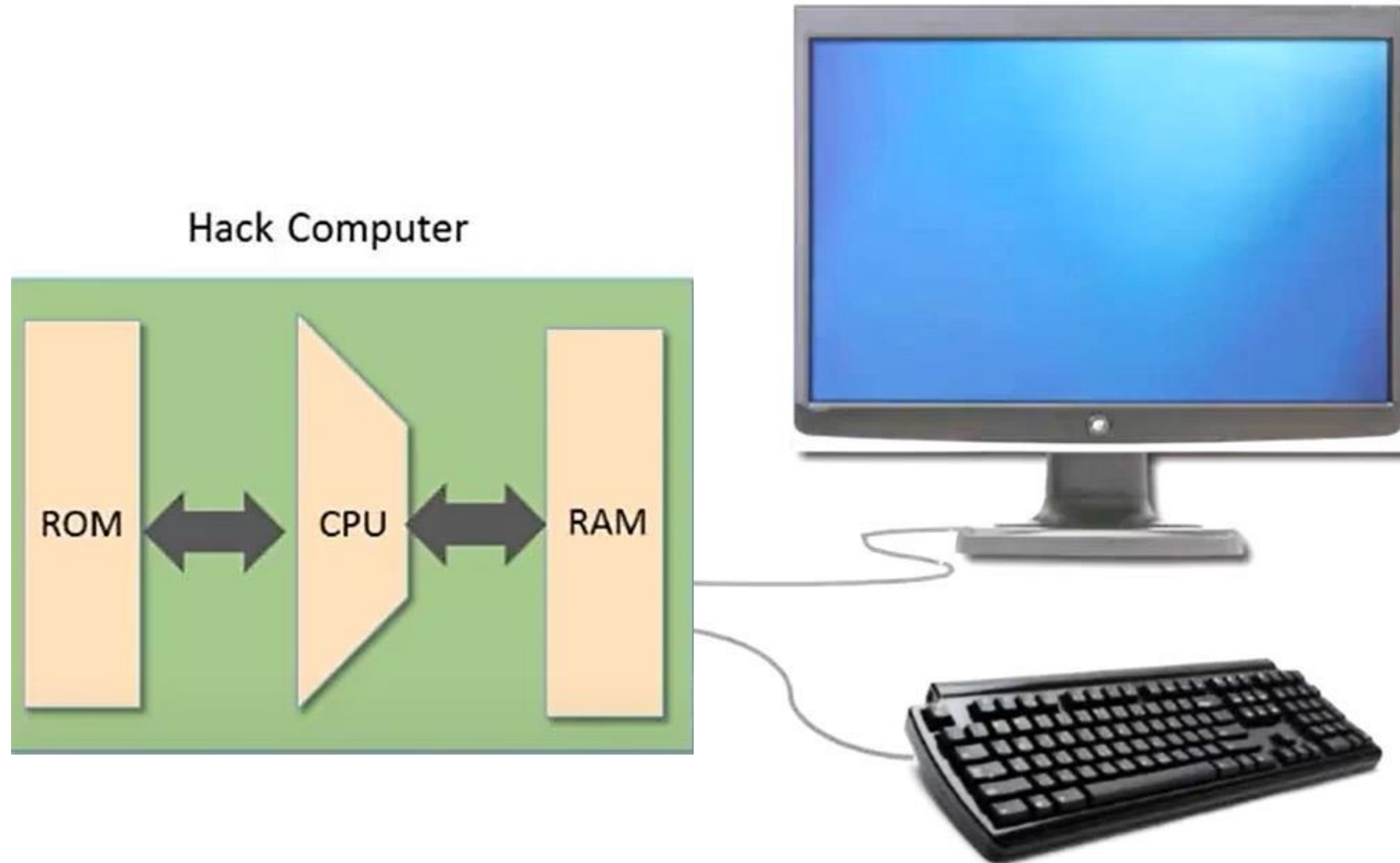


Computing System

- ❑ A combination of **hardware** (physical components) and **software** (programs that control them).
- ❑ **Found everywhere**: laptops, smartphones, cars, appliances, cloud servers.
- ❑ **Why it matters**: Understand how information is **represented**, **processed**, **stored**, and **controlled**.

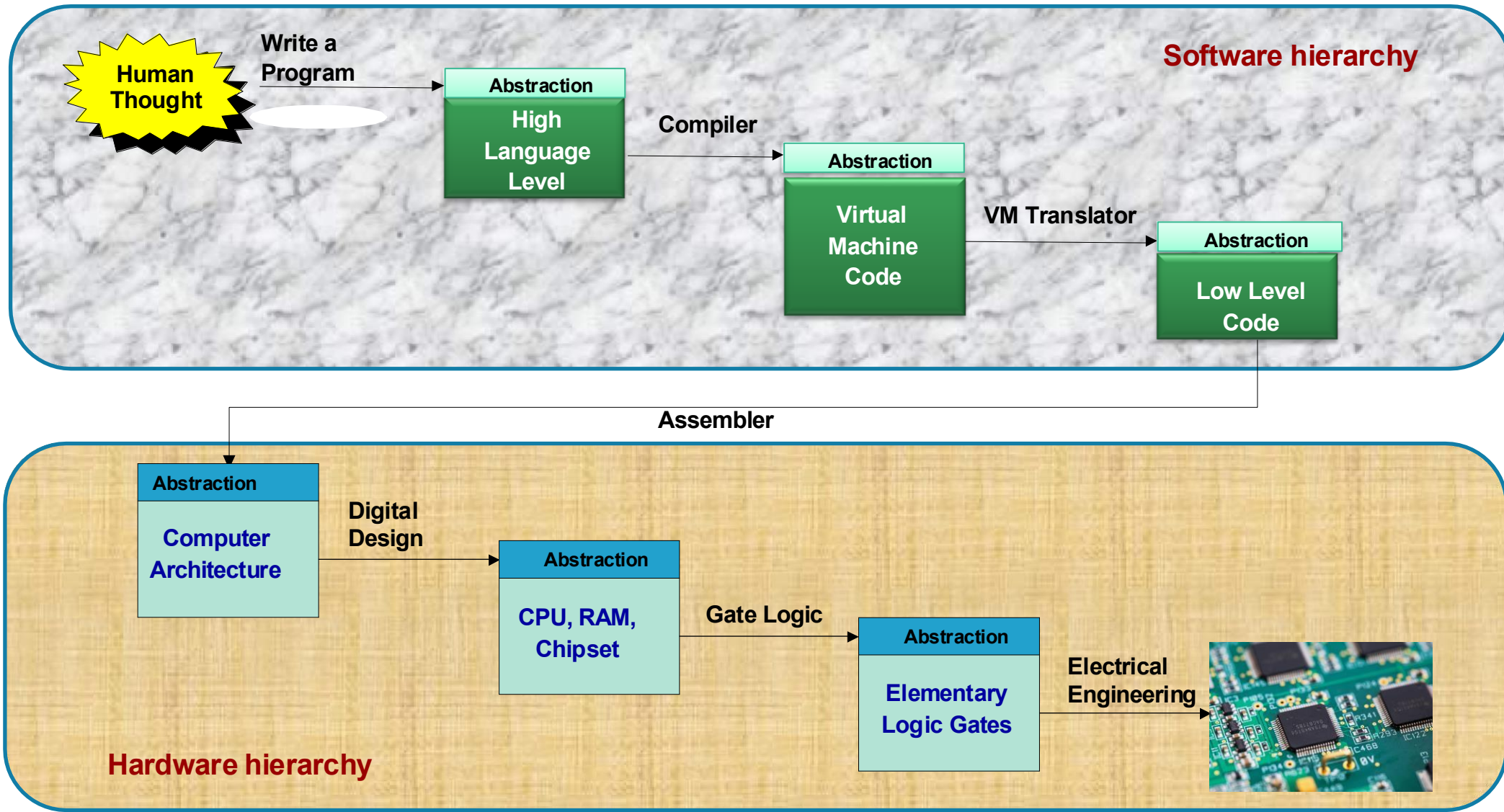


Hack Computer

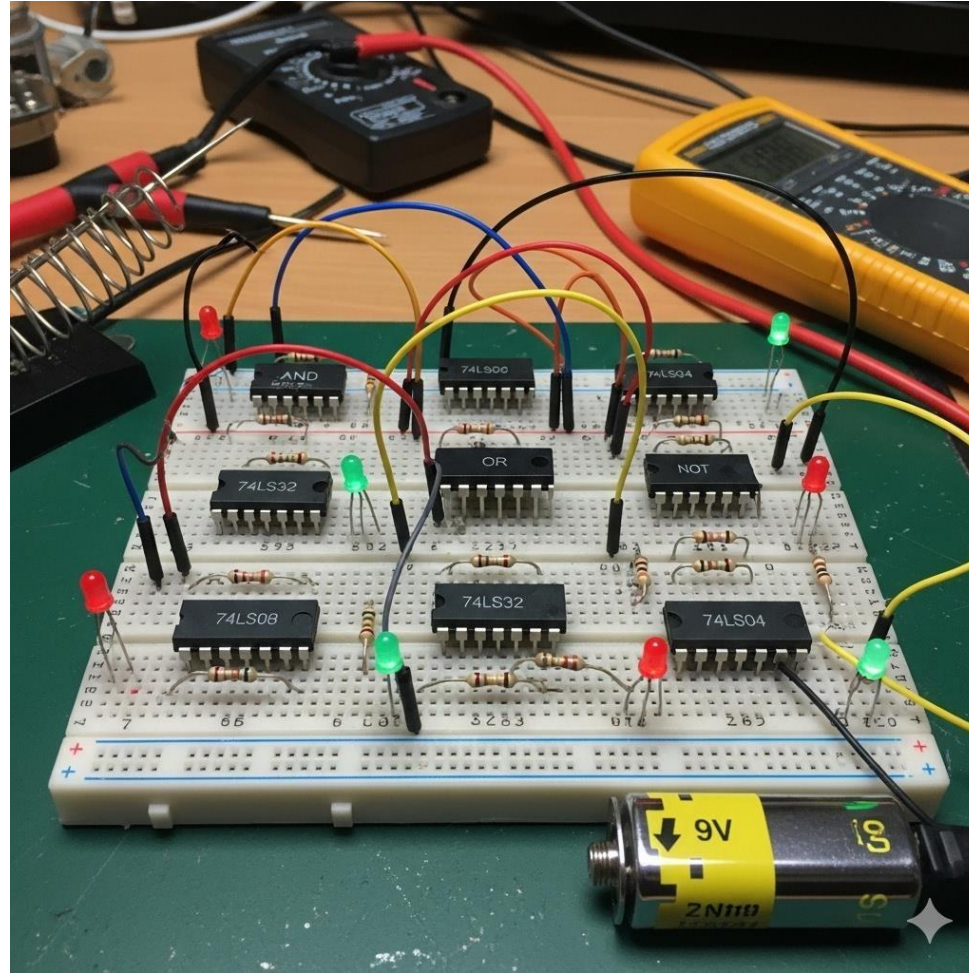


Why Take This Course?

- Most courses teach top-down: "write code, get output."



Why Take This Course?



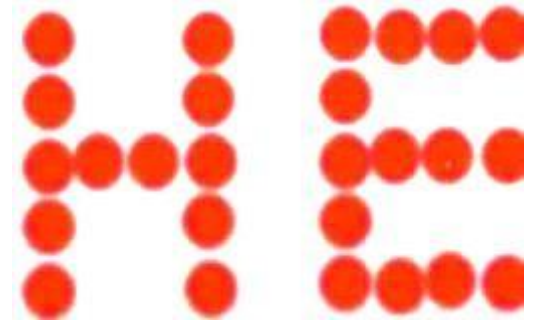
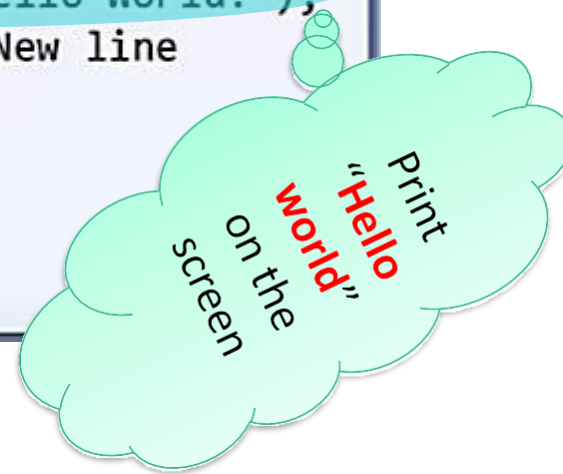
Why Take This Course?

```
// First Example in Programming 101
Class Main {
  function void main() {
    do Output.printString("Hello World!");
    do Output.println(); // New line
    return;
  }
}
```

p=112

r=114

i=105



input

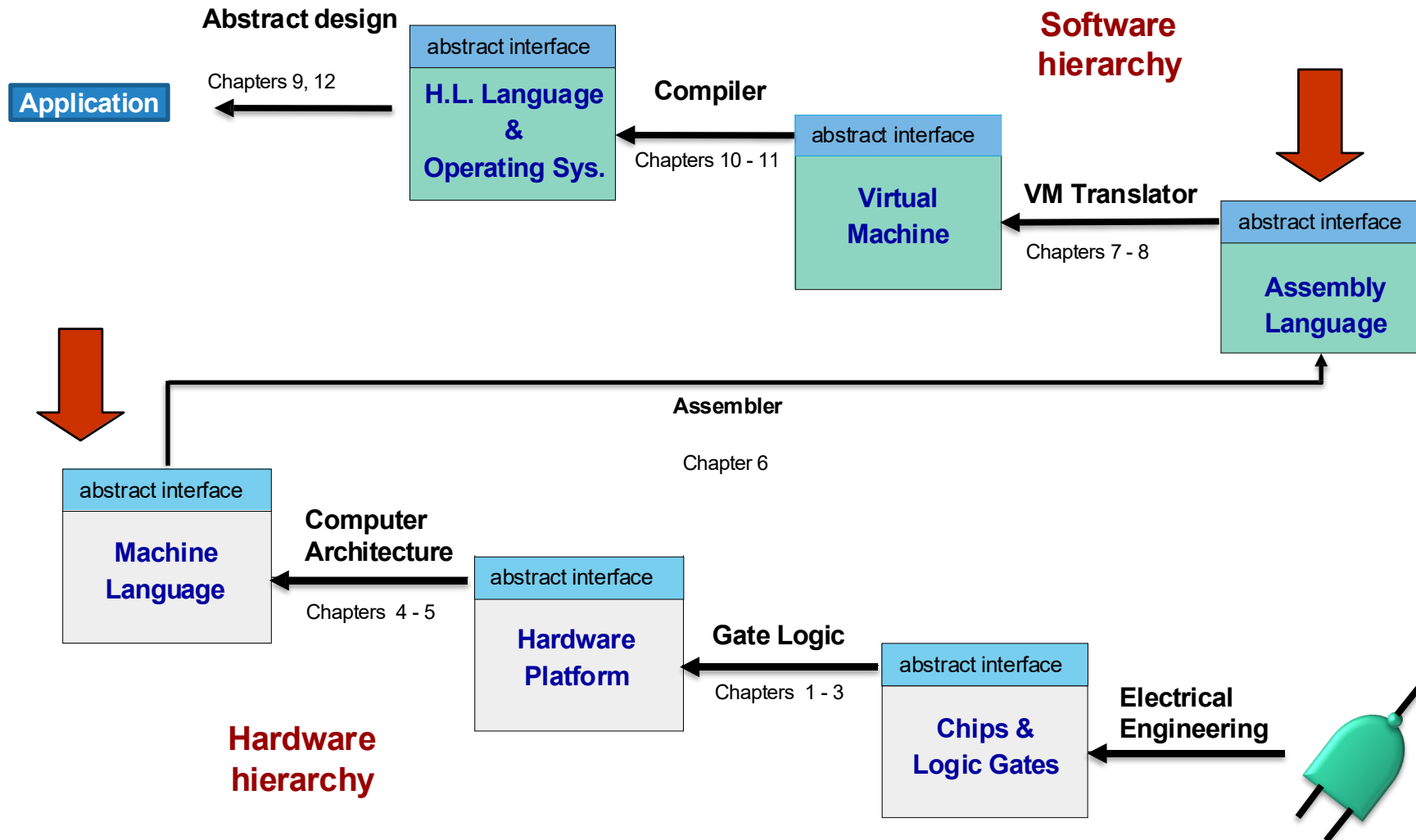
Black box

output

Why Take This Course?

- ❑ **This course:** We go bottom-up: “build the machine that runs the code.” no black boxes:
 - ❑ Start with a **NAND** gate → end with a working computer.
 - ❑ Build every layer yourself (hardware and software).
- ❑ **Goal:** Understand how hardware and software interact how they are really built — no black boxes/ not just how to use them.

Why Take This Course?



Course Objectives

- ❑ Understand how hardware and software systems are constructed, and how they work together
- ❑ Learn to **break complex problems into simpler components.**
- ❑ Discover how **large-scale systems and projects** are planned and built
- ❑ **Have fun!** — build your own computer step by step!

Methodology

- ❑ **Build** a complete, general-purpose, working computer system.
- ❑ **Experiment and play** with every layer of this computer:
 - ❑ Hardware → Architecture → Software → Applications
- ❑ **Learn by doing:** projects make theory tangible
- ❑ Automated tests verify your work

What You Will Learn

- ❑ **Digital Hardware:** Boolean logic, circuits (combinational & sequential), memory.
- ❑ **Computer Architecture:** ALU, CPU, machine language.
- ❑ **Software Stack:** assembler, virtual machine (VM), compiler, Operating Systems (OS).
- ❑ **Design Principles:** abstraction vs. implementation, modularity, testing.
- ❑ **Tools:** HDL simulators, CPU/VM emulators, self-contained build environment.

Course Roadmap

□ Part I - Hardware (Projects 1-6):

NAND → Logic gates → ALU → Memory → CPU → Assembler

□ Part II - Software (Projects 7-12):

VM → Compiler → OS → High-level/Jack applications (like Tetris!)

□ Unit 0 Today: course overview, abstraction, and tools.

Abstraction & Modularity

- **Abstraction:** What a module does *vs* how it works inside
- **Modularity:** Build/test components separately, then integrate
- **Layered learning:** small projects → combine → full system

Building a chip: Design

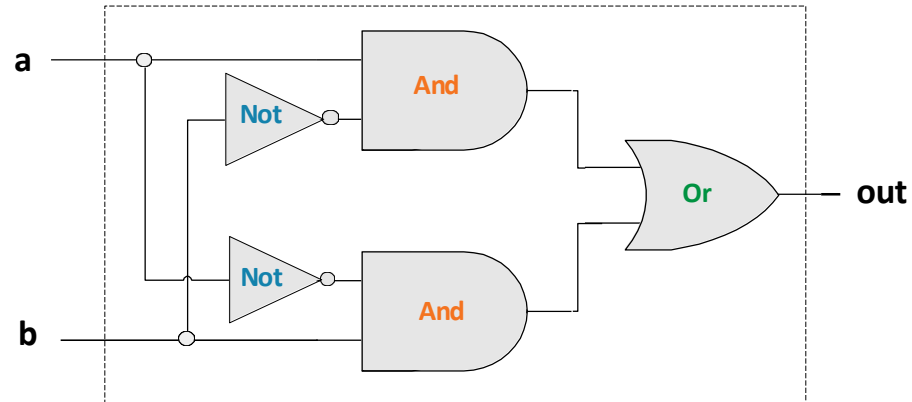
Chip Abstraction

Xor: outputs 1 if exactly one of its inputs is 1, otherwise outputs 0.

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

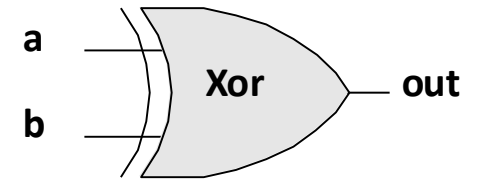


Modular Design



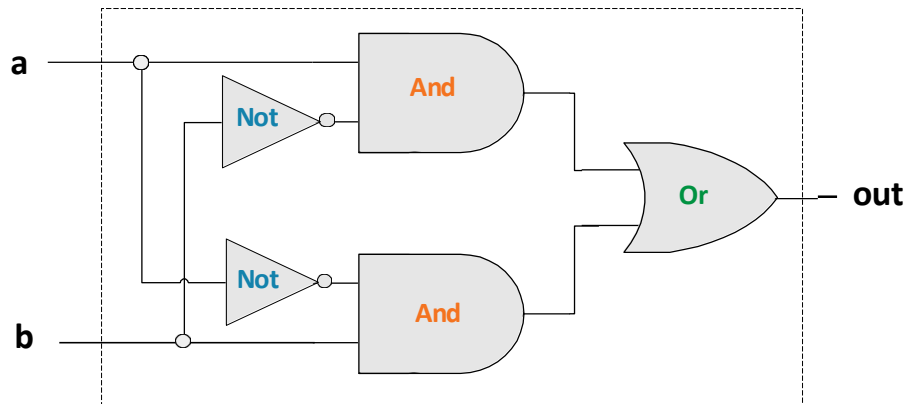
$$\text{Xor}(a,b) = \text{Or}(\text{And}(a,\text{Not}(b)),\text{And}(\text{Not}(a),b))$$

Interface



Building a chip: Design

Design



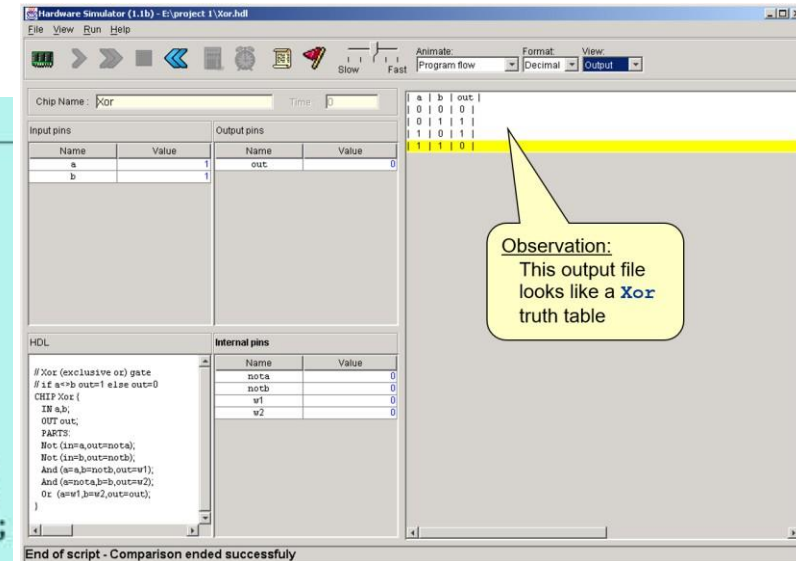
Specify

HDL program (Xor.hdl)

```
/* Xor (exclusive or) gate:
   If a!=b out=1 else out=0. */
CHIP Xor {
  IN a, b;
  OUT out;
  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=w1);
    And (a=nota, b=b, out=w2);
    Or (a=w1, b=w2, out=out);
}
```

Test script (Xor.tst)

```
load Xor.hdl,
output-list a, b, out;
set a 0, set b 0,
eval, output;
set a 0, set b 1,
eval, output;
set a 1, set b 0,
eval, output;
set a 1, set b 1;
eval, output;
```



Tools You'll Use

- ❑ **Nand2Tetris software suite (open source):**

- ❑ Hardware simulator → test HDL chips
- ❑ CPU emulator → run Hack programs
- ❑ VM emulator → test stack ops/functions
- ❑ Assembler and Jack compiler

- ❑ **No prior hardware knowledge required**

Course Philosophy

- **Hands-on first:** you build every chip and tool
- **Minimal prerequisites:** just basic programming
- **Fast but focused:** concepts > industrial complexity
- **Immediate feedback:** test scripts confirm correctness

First Step

- Unit 0: First Step
- Set up environment and tools
- Understand **hardware-software abstraction layers**
- Try HDL and simulation (no soldering required!)
- Prepare for **Project 1: Logic gates from NAND**

Big Takeaways

- By building one yourself, you'll:
 - Understand the **hardware-software boundary**
 - Learn principles that scale to any system
 - Gain confidence to tackle complex problems

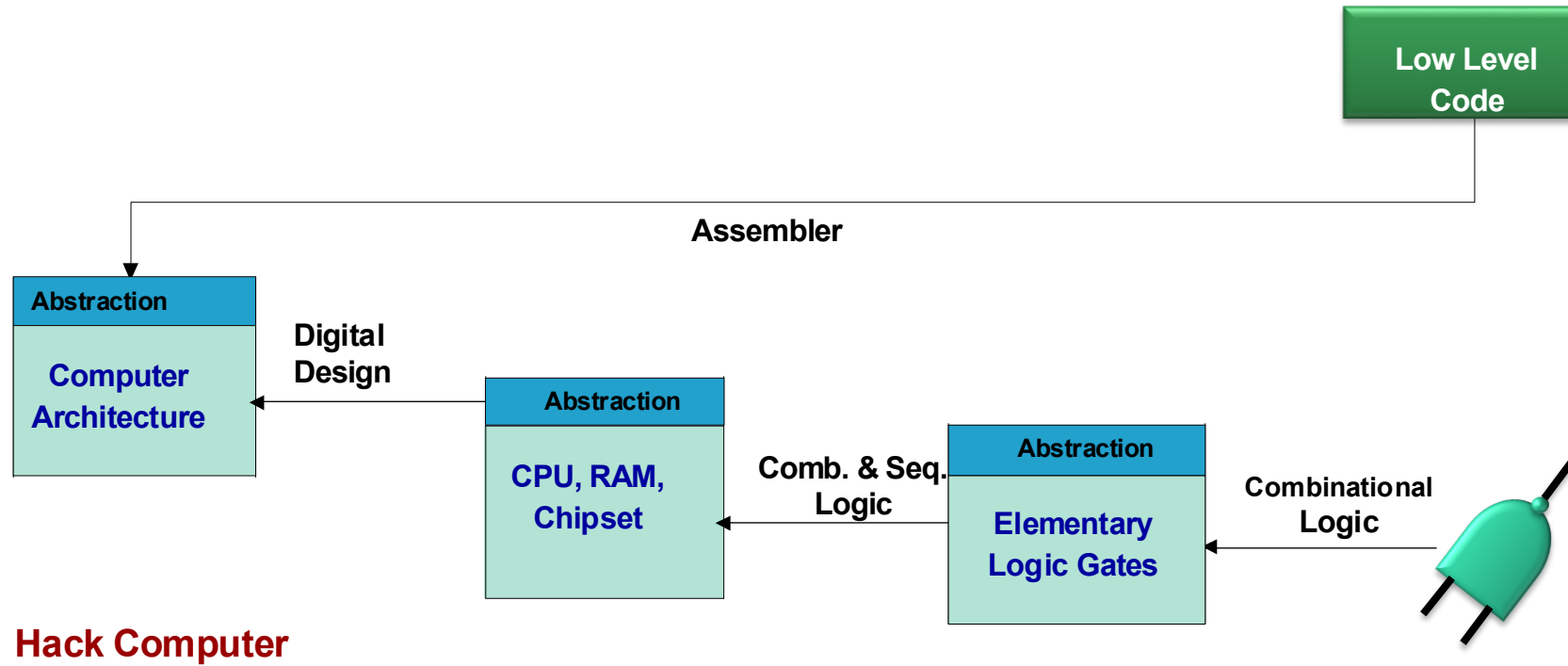
Grading Policy

- 20% Class Attendance
- 20% Assignments
- 20% Mid-Term Exam
- 20% Final Exam
- 20% Final Project (Integration & Presentation)

Expectations from Students

- Regular attendance
- Hands-on practice with projects
- Submit assignments on time
- Group discussions

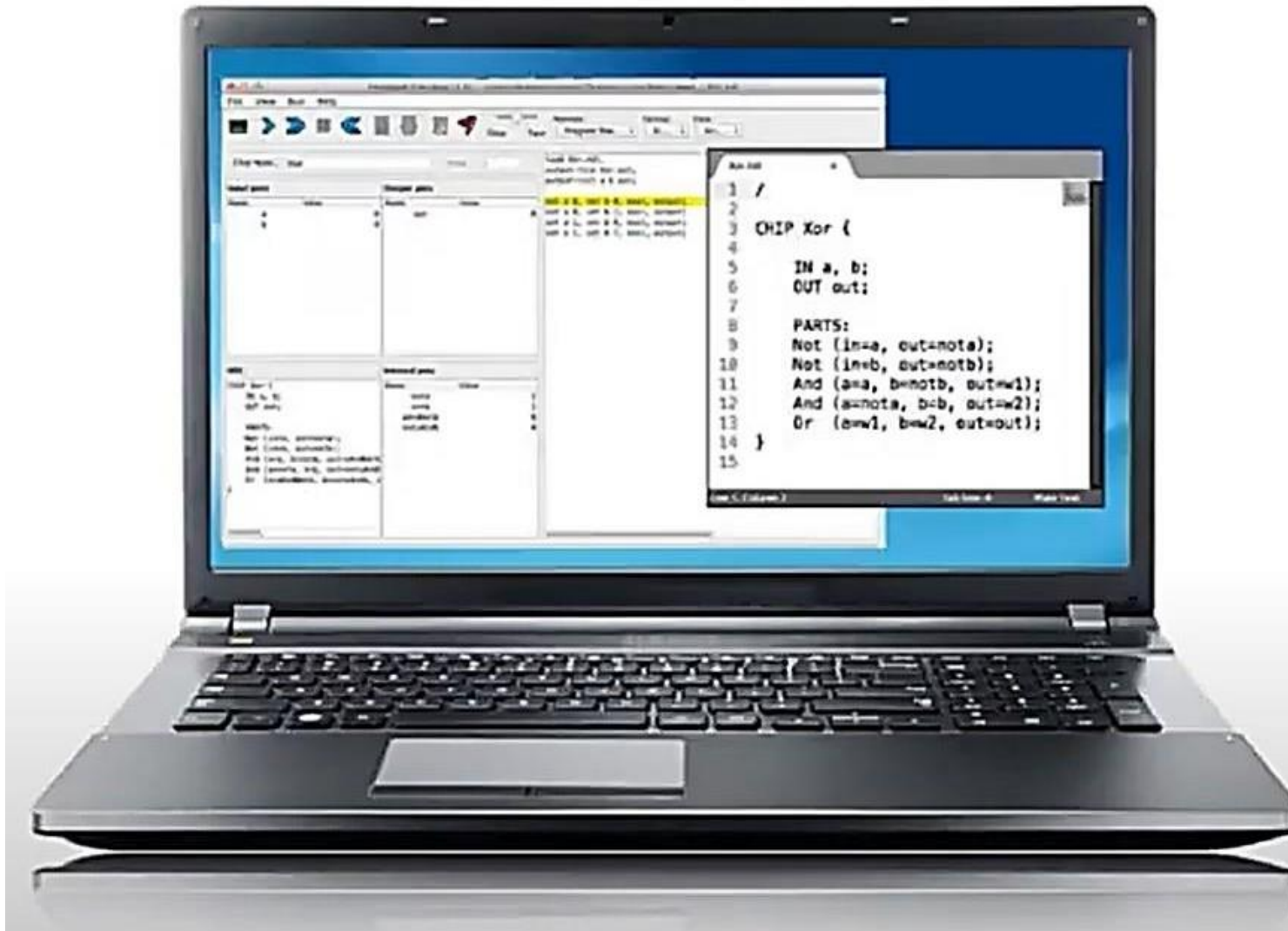
Part 1



How to build a chip?



How to build a chip?



Weekly Projects

Week 1

Elementary logic gates

- Nand (primitive)
- Not
- And
- Or
- Xor
- Mux,
- Dmux
- Not16
- And16
- Or16
- Mux16
- Or8Way
- Mux4Way16
- Mux8Way16
- DMux4Way
- DMux8Way

Week 2

Arithmetic- Logic Unit

- HalfAdder
- FullAdder
- Add16
- Inc16
- ALU

Week 3

Registers and memory

- Bit
- Register
- RAM8
- RAM64
- RAM512
- RAM4K
- RAM16K
- PC

Week 5

Computer architecture

- Memory
- CPU
- Compute

Week 4

Writing low- level programs

Week 6

Developing an assembler



